ÉTUDE DE CAS — STRIPE

LIVRABLE 2

ERD pour le système OLTP

Certification RNCP 41993 – Adrien COSSON
Bloc 2 - Concevoir et déployer l'infrastructure de données et de calcul pour l'IA

Ce livrable présente le modèle de données de la base transactionnelle (OLTP) de Stripe, déployée sur Amazon Aurora PostgreSQL. Cette base enregistre chaque paiement, remboursement, rétrofacturation et abonnement au moment où il se produit. C'est la source de référence pour tous les autres systèmes de l'architecture (livrable 1).

1. Diagramme entité-relation

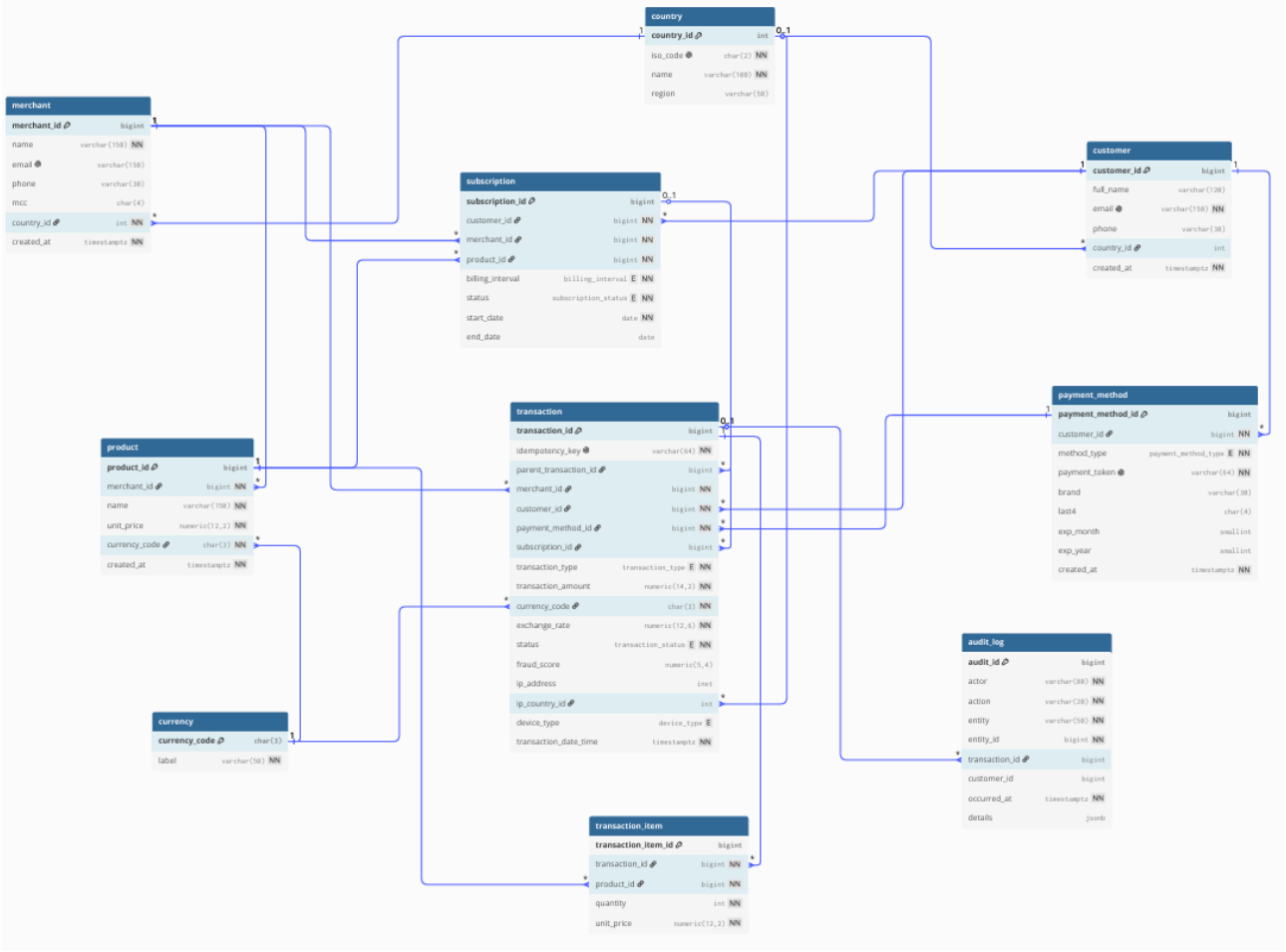


Figure 1 – Diagramme entité-relation de la base OLTP (dbdiagram.io)

Lecture du diagramme : l'icône en forme de clé indique la clé primaire, NN un champ obligatoire et E une énumération (liste de valeurs autorisées). Sur chaque relation, « 1 » indique le côté « un », « * » le côté « plusieurs » et « 0..1 » une relation facultative.

2. Les tables du modèle

Table	Contenu	Remarque
transaction	Une ligne par opération : paiement, remboursement, rétrofacturation ou paiement d'abonnement	Table centrale. Un remboursement ou une rétrofacturation est une nouvelle ligne reliée au paiement d'origine (parent_transaction_id)
transaction_item	Les produits achetés dans une transaction (quantité, prix unitaire)	Permet d'analyser la performance des produits
merchant	Les marchands qui utilisent Stripe	Données de contact du marchand
customer	Les clients finaux	Seule table, avec merchant, qui contient des données personnelles (nom, email, téléphone)
payment_method	Les moyens de paiement d'un client	Le numéro de carte n'est jamais stocké : seulement un jeton/token (payment_token) et les 4 derniers chiffres
product	Le catalogue des produits de chaque marchand	Données de référence
subscription	Les abonnements d'un client chez un marchand	Une transaction de type « subscription » peut y être reliée
country	Les pays et leur région	Données de référence, utilisées pour le marchand, le client et le pays de l'adresse IP
currency	Les devises	Données de référence
audit_log	Le journal des actions sur les données (qui, quoi, quand)	Sert à la traçabilité et à la conformité (livrable 6)

Les champs demandés par le sujet pour les données transactionnelles sont tous présents dans la table transaction : identifiants de la transaction, du marchand et du client, montant, devise, moyen de paiement, date et heure, localisation par l'adresse IP (ip_address et ip_country_id), type d'appareil, statut et score de fraude.

3. Normalisation

Le modèle est normalisé (troisième forme normale) : chaque information n'est stockée qu'à un seul endroit. Par exemple, le nom d'un pays n'est écrit que dans la table country, et les autres tables ne contiennent que son identifiant. Une modification ne se fait donc qu'une fois, sans risque d'avoir deux valeurs différentes pour la même information.

Deux exceptions sont volontaires, car elles décrivent la situation au moment du paiement et ne doivent plus changer ensuite :

- **Le taux de change (exchange_rate) :** il est enregistré sur la transaction, car le taux du jour du paiement doit être conservé même si le taux évolue ensuite.
- **Le prix unitaire (unit_price) :** il est enregistré sur la ligne de transaction_item, car le prix d'un produit peut changer après l'achat.

4. Intégrité transactionnelle (ACID)

Les propriétés ACID garantissent qu'une opération sur la base est fiable, même en cas de panne ou d'accès simultanés.

Atomicité : une opération est entièrement validée ou entièrement annulée. La transaction, ses produits (transaction_item) et la ligne d'audit_log sont écrits dans une seule transaction SQL.

Cohérence : la base passe toujours d'un état valide à un autre. Les clés étrangères, les champs obligatoires, les énumérations et la clé d'idempotence unique empêchent toute écriture qui ne respecte pas ces règles.

Isolation : deux opérations simultanées ne se perturbent pas. Par exemple, deux remboursements lancés en même temps sur le même paiement ne peuvent pas dépasser son montant, car PostgreSQL les traite l'un après l'autre.

Durabilité : une opération validée n'est jamais perdue, même après une panne. Chaque écriture est d'abord inscrite dans le journal des modifications (WAL), et le stockage d'Aurora est copié 6 fois sur 3 zones.

- **Transactions SQL :** l'enregistrement d'un paiement écrit plusieurs lignes en même temps (la transaction, ses produits dans transaction_item et la ligne d'audit_log). Ces écritures sont regroupées dans une seule transaction SQL : soit tout est validé, soit rien ne l'est. Une panne au milieu de l'écriture ne laisse donc jamais un paiement à moitié enregistré.
- **Pas de double prélèvement :** la clé d'idempotence (idempotency_key) est unique. Si le marchand renvoie la même demande de paiement, par exemple après une coupure réseau, la base refuse la deuxième écriture et renvoie le résultat de la première.
- **Contraintes :** les clés étrangères empêchent une transaction de pointer vers un client ou un marchand qui n'existe pas, les champs obligatoires (NN) ne peuvent pas être vides et les énumérations n'acceptent que les valeurs prévues (par exemple : successful, failed, refunded pour le statut).
- **Historique complet :** un paiement n'est jamais modifié pour être remboursé : le remboursement est une nouvelle ligne qui pointe vers le paiement d'origine. On garde ainsi l'historique de toutes les opérations, ce qui est indispensable pour une donnée financière (livrable 8, requête 10).

5. Performances

- **Index :** en plus des clés primaires, des index sont créés sur (customer_id, transaction_date_time) et (merchant_id, transaction_date_time), qui correspondent aux recherches les plus fréquentes (paiements récents d'un client ou d'un marchand), sur parent_transaction_id pour retrouver les remboursements, et sur customer_id dans audit_log pour les demandes RGPD.
- **Séparation des lectures et des écritures :** les écritures de paiement vont sur l'instance principale, tandis que les lectures (support client, équipe fraude) vont sur les réplicas. Les analyses lourdes sont faites dans Redshift (livrable 3), jamais dans l'OLTP.
- **Taille de la base :** les transactions de plus de 24 mois sont retirées de la base chaque mois. Elles restent disponibles dans le Data Lake, dans Redshift et dans S3 Glacier (livrable 5).
- **Montée en charge :** si une seule instance d'écriture ne suffit plus, la table transaction peut être fragmentée (sharding) par marchand sur plusieurs machines avec Aurora PostgreSQL Limitless Database (livrable 1, section 7).

6. Réplication et basculement

Mécanisme	Fonctionnement	Objectif
Stockage répliqué	Chaque donnée écrite est copiée 6 fois sur 3 zones de disponibilité par Aurora	Aucune transaction validée n'est perdue si un disque ou un centre de données tombe en panne
Réplicas en lecture	2 réplicas dans deux autres zones, mis à jour en quelques millisecondes	Absorber les lectures et prendre le relais en cas de panne
Basculement automatique (failover)	Si l'instance principale tombe, un réplica est promu automatiquement	Reprise des écritures en environ 30 secondes
Copie dans une autre région	Aurora Global Database copie la base en continu dans la région de Francfort	Reprise en quelques minutes en cas de sinistre sur toute une région, avec une perte de l'ordre de la seconde
Sauvegardes continues	Sauvegarde automatique conservée 35 jours	Revenir à un instant précis après une erreur humaine
Réplication logique (CDC)	Debezium lit le journal des modifications (WAL) et publie chaque changement dans Kafka	Synchroniser les autres systèmes en temps réel sans ralentir la base (livrable 5)

7. Sécurité et données personnelles

- **Données de carte :** aucun numéro de carte n'est stocké, seulement le jeton/token et les 4 derniers chiffres. La base reste ainsi en dehors de la partie la plus sensible du périmètre PCI-DSS (livrable 6).
- **Données personnelles regroupées :** le nom, l'email et le téléphone ne se trouvent que dans customer et merchant. Pour une demande d'effacement RGPD, il suffit d'anonymiser ces colonnes : les transactions sont conservées pour des raisons légales, mais ne permettent plus d'identifier la personne.

- **Traçabilité :** chaque action sur les données est enregistrée dans audit_log (qui, quelle action, sur quelle table et quelle ligne, quand). Le champ customer_id permet de retrouver toutes les actions concernant un client (livrable 8, requête 11).